МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет по курсовому проекту по дисциплине "Функциональное программирование" Реализация проекта на языке Haskell по теме "Визуализатор маршрутов на графах"

Отчет выполнил
студент группы 5130201/40001 _____ Костина А.В.
Отчет принял
преподаватель _____ Моторин Д.Е.

«____»_____ 2026г.

Санкт-Петербург, 2026

Содержание

Техническое задание **3**

7. Результаты работы **10**

Введение **11**

1 Постановка задач **12**

2 Особенности реализации **13**
2.1 Описание разработки проекта 13
2.2 Описание технических особенностей реализации проекта 13

3 Примеры работы программы **14**

4 Руководство по эксплуатации для пользователя **15**

Заключение **16**

Список литературы **17**

Приложение **18**
A.1 18

Техническое задание

1. Основание для выполнения работы

Работа выполняется в соответствии с учебным планом по дисциплине «Функциональное программирование» в рамках курсового проектирования студента группы 5130201/40001 Института компьютерных наук и кибербезопасности Федерального государственного автономного образовательного учреждения высшего образования «Санкт-Петербургский политехнический университет Петра Великого», обучающегося по направлению подготовки 02.03.01 Математика и компьютерные науки.

2. Исполнитель и соисполнители работы

Исполнитель: студент группы 5130201/40001
Соисполнители: отсутствуют

3. Цели, задачи и исходные данные

a. Цель работы

Создание наглядного инструмента для поиска и визуализации кратчайших маршрутов в транспортных сетях (метрополитен, железные дороги), позволяющего пользователю загружать схему сети в формате JSON, задавать начальную и конечную станции, получать оптимальный маршрут и его графическое представление.

b. Задачи работы

В процессе выполнения работы требуется решить следующие задачи:

- **Чтение и запись файлов:**
  - реализовать импорт графа из JSON-файла, описывающего схему метро/ЖД с использованием библиотеки `aeson`;
  - реализовать экспорт графа в JSON-файл;
  - реализовать сохранение визуализаций графа и найденного кратчайшего пути в PNG-файлы.
- **Обработка информации математическими функциями:**

  - разработать структуры данных для представления взвешенных графов (ориентированных/неориентированных);
  - реализовать алгоритм Дейкстры для поиска кратчайшего пути;
  - реализовать восстановление маршрута от начальной до конечной вершины.

- **Взаимодействие с пользователем через консоль:**
  - разработать систему меню для управления программой;
  - обеспечить ввод начальной и конечной станций для построения маршрута.

- **Логирование действий пользователя:**
  - реализовать запись в лог-файл всех значимых действий;
  - добавить временные метки и уровни логирования.

- **Обработка ошибок:**
  - обеспечить корректную обработку ошибок ввода/вывода;
  - обрабатывать некорректный пользовательский ввод;
  - обрабатывать ситуации, когда путь между станциями отсутствует.

c. Исходные данные

Исходными данными для проведения работы являются:

- JSON-файлы с описанием графов транспортных сетей в формате, включающем названия станций и перегоны с весами;
- готовые датасеты транспортных сетей (например, из открытых источников);

4. Основное содержание работы

a. Разработка проекта

Разработка проекта включает:

- проектирование архитектуры и системы типов;
- реализацию модулей для работы с графами, алгоритмом Дейкстры, визуализацией, вводом/выводом JSON, логированием и обработкой ошибок;

- написание тестов с использованием QuickCheck;
- отладку и тестирование программы.

b. Отчетная документация

Подготовка отчета, включающего титульный лист, содержание, ТЗ, введение, описание разработки, примеры работы, руководство пользователя, заключение, список литературы и приложения.

5. Основные требования к выполнению работы

a. Назначение разрабатываемого программного комплекса

Программный комплекс предназначен для поиска и визуализации кратчайших маршрутов в транспортных сетях (метро, ЖД), представленных в формате JSON. Может использоваться как вспомогательный инструмент для планирования поездок, а также в образовательных целях при изучении алгоритмов на графах.

b. Состав разрабатываемого программного комплекса

- исполняемый модуль;
- модули исходного кода на Haskell;
- JSON-файлы с примерами схем метро и ЖД;
- документация пользователя;
- файлы логов и визуализаций.

c. Технические характеристики

- Язык реализации: Haskell;
- Система сборки: Stack;
- Операционная система: Windows;

- Взаимодействие: консольный ввод/вывод;
- Визуализация: генерация PNG-файлов с использованием библиотеки `graphite`;
- Парсинг JSON: библиотека `aeson`.

d. Функциональные требования

d.1. Импорт и экспорт файлов

- Импорт схемы метро/ЖД из JSON-файла со следующей структурой:

```
{
 "type": "metro",
 "name": "Московский метрополитен",
 "stations": [
 {"id": 1, "name": "Библиотека имени Ленина"},
 {"id": 2, "name": "Арбатская"},
 {"id": 3, "name": "Александровский сад"}
 ],
 "edges": [
 {"from": 1, "to": 2, "weight": 2.5},
 {"from": 2, "to": 3, "weight": 1.8},
 {"from": 1, "to": 3, "weight": 4.0}
 ]
}
```

- Экспорт текущей схемы в JSON-файл.
- Проверка корректности JSON-файла при импорте (валидация структуры, типов данных, ссылочной целостности).

d.2. Навигация по станциям

- Отображение списка всех доступных станций с их номерами и названиями.
- Поиск станции по названию (частичному совпадению).
- Выбор начальной и конечной станции из списка (по номеру или названию).

d.3. Поиск кратчайшего пути (алгоритм Дейкстры)

- Запуск от указанной стартовой станции до указанной конечной станции.

- Отображение кратчайшего расстояния (времени в пути).
- Восстановление и отображение полной последовательности станций маршрута.
- Сохранение итоговой визуализации маршрута в PNG-файл с выделением найденного пути.

d.4. Визуализация

- Сохранение текущего состояния схемы в PNG-файл.
- Сохранение найденного кратчайшего маршрута в PNG-файл с выделением пути.
- Цветовое кодирование состояний:
  - станции, не входящие в маршрут — белый фон, чёрная граница;
  - станции, входящие в итоговый маршрут — синий фон;
  - ребро, входящее в итоговый маршрут — синяя линия (жирная);
  - остальные рёбра — серая линия.
- Подписи станций (названия) и рёбер (время в пути).

d.5. Консольный интерфейс

Главное меню:

```
========== ГЛАВНОЕ МЕНЮ ==========
1. Загрузить схему из файла
2. Выбрать станции для маршрута
3. Найти кратчайший маршрут
4. Визуализация
5. Просмотр лога
6. Выход
Ваш выбор:
```

Меню "Выбрать станции для маршрута":

- Показать список всех станций
- Найти станцию по названию
- Выбрать начальную станцию

- Выбрать конечную станцию
- Показать выбранные станции
- Назад

Меню "Найти кратчайший маршрут":

- Запустить поиск
- Показать результат (расстояние и маршрут)
- Сохранить визуализацию маршрута
- Назад

Меню "Визуализация":

- Сохранить текущее состояние схемы
- Сохранить последний найденный маршрут
- Выбрать цветовую схему
- Назад

Меню "Просмотр лога":

- Показать последние N записей
- Показать все записи
- Очистить лог
- Назад

d.6. Логирование

- Файл лога: `graph_app.log`
- Формат записи: `[YYYY-MM-DD HH:MM:SS] [LEVEL] Сообщение`
- Уровни логирования: INFO, WARNING, ERROR
- Логируемые события:
  - запуск и завершение программы;
  - загрузка и сохранение схемы;
  - выбор начальной и конечной станций;
  - запуск поиска маршрута;
  - результаты поиска (найден/не найден, длина пути);
  - ошибки при работе с JSON-файлами;
  - ошибки пользовательского ввода.

d.7. Обработка ошибок

- **Ошибки ввода-вывода:**
  - файл не найден — вывод сообщения "Файл не найден. Проверьте путь и имя файла";
  - некорректный JSON — вывод сообщения с указанием ошибки парсинга;
  - ошибка при сохранении — вывод причины ошибки.
- **Ошибки пользовательского ввода:**
  - ввод нечисловых значений — повтор запроса с сообщением "Ожидается число";
  - указание несуществующей станции — вывод списка существующих станций;
  - отрицательные веса — сообщение о недопустимости.
- **Ошибки алгоритма Дейкстры:**
  - отсутствие пути между станциями — сообщение "Маршрут не найден";
  - пустая схема — сообщение "Схема не содержит станций".

e. Стандартные средства

При разработке должны использоваться типовые средства языка Haskell и существующие библиотечные решения:

- `aeson` — для парсинга и генерации JSON-файлов;
- `containers` (Data.Map, Data.Set, Data.Sequence) — для представления графов и структур данных алгоритмов;
- `graphite` — для визуализации графов и сохранения в PNG;
- `QuickCheck` — для тестирования свойств;
- `directory` — для работы с файловой системой;
- `time` — для формирования временных меток в логах;
- `bytestring` — для эффективной работы с JSON.

6. Этапы и сроки выполнения работы



Рис. 1: Диаграмма Ганта этапов работ

7. Результаты работы

1. Утверждённое техническое задание.
2. Диаграмма модулей, описание типов данных, обоснование выбора библиотек.
3. Работающие модули парсинга JSON, поиска маршрута (алгоритм Дейкстры), визуализации, консольного интерфейса, логирования и обработки ошибок.
4. Готовый отчёт с описанием разработки, примерами работы и руководством пользователя.

Введение

1 Постановка задач

2 Особенности реализации

2.1 Описание разработки проекта

2.2 Описание технических особенностей реализации проекта

3 Примеры работы программы

4 Руководство по эксплуатации для пользователя

Заключение

Список литературы

[1]

Приложение

A.1